

El examen es individual. Lea con detenimiento y tranquilidad cada uno de las preguntas y planteamientos. Justifique sus respuestas de manera clara y resumida. Respire y tómese su tiempo para responder y argumentar.

1. Verdadero o Falso (5 puntos)

Responda si está seguro, se aplica factor de corrección, es decir, una mala elimina una buena.

1. La cohesión se define como la relación funcional entre los elementos de una clase o módulo. Cuanto menor sea la cohesión, mayor será la posibilidad de que una clase infrinja el Principio SRP.
(☐) Verdadero (☐) Falso
2. Seemann en su propuesta de implementar AOP con SOLID, corrige la violación del Principio OCP haciendo uso del patrón de *refactoring* denominado *Parameter Object*.
(☐) Verdadero (☐) Falso
3. Seemann en su propuesta de implementar AOP con SOLID, corrige la violación del Principio LSP usando una clase abstracta.
(☐) Verdadero (☐) Falso
4. Las entidades en DDD representan a las principales abstracciones del dominio y se caracterizan por identidades y ciclos de vida. Las entidades tienen comportamientos y encapsulan reglas de negocio relacionadas con su estado e interacciones con otros objetos del dominio.
(☐) Verdadero (☐) Falso
5. Los *value objects* representan características y atributos esenciales para el dominio. No tienen identidad y son inmutables. Los *value objects* pueden ser compartidos por varias entidades/agregados. El cambio de estado de un atributo cuyo tipo es un *value object*, se hace alterando el mismo *value object*.
(☐) Verdadero (☐) Falso
6. Los servicios de dominio encapsulan procesos o políticas del negocio, o también representan algún contrato a nivel de la lógica de negocio. Ellos representan algún comportamiento, no tienen identidad, son *stateless*, con frecuencia colaboran con múltiples entidades, objetos del dominio y mediante puertos pueden orquestar responsabilidades con la capa de aplicación.
(☐) Verdadero (☐) Falso
7. Ningún objeto de dominio fuera del límite del agregado puede contener una referencia a objetos internos. El *Aggregate Root* puede devolver copias de objetos internos para su uso en operaciones de dominio y cualquier cambio en estas copias de objetos internos se actualizará automáticamente en el *Aggregate*.
(☐) Verdadero (☐) Falso

8. Un servicio de dominio gestiona cualquier operación relacionada con un concepto de dominio que no forma parte inherente de ningún agregado del dominio. A diferencia de los servicios de aplicación que emplean DTOs para la comunicación, un servicio de dominio opera directamente con entidades de dominio.
☐ Verdadero ☐ Falso
9. El modelo de dominio no pretende ser indulgente, sino que debe lanzar alarmas cuando algo se desvíe del comportamiento esperado. En cuanto a los mensajes de error, un modelo de dominio debe definir su propio conjunto de excepciones (excepciones de dominio) y hacerlas accesibles a las capas superiores, como la capa de aplicación.
☐ Verdadero ☐ Falso
10. La solución propuesta por el Profesor Carlos Alonzo supera a la de Seemann, porque mediante el uso de la mónada *Either* se logra manejar de manera elegante los valores de retorno de servicios y las excepciones (de dominio, aplicación o infraestructura) evitando lanzar excepciones que rompan el flujo del programa de manera imprevista.
☐ Verdadero ☐ Falso

2. Demuestre su aporte en el proyecto (3 puntos)

Usted debe mostrar sus principales abstracciones por cada capa de la arquitectura que usó en su proyecto (backend o frontend). Debe escoger solamente un (1) caso de uso que usted haya implementado. Evidentemente los nombres de las clases y sus relaciones entre las capas deben ser muy similares a su aporte en el proyecto. No tiene que mostrar métodos, solo los nombres de sus abstracciones (clases concretas, clases abstractas, interfaces).

<i>Capa:</i>	<i>Capa:</i>	<i>Capa:</i>

3. Modelo de Dominio con *Domain-driven Design/DDD* (5 puntos)

Estás diseñando un sistema para una empresa que organiza eventos corporativos y sociales. La empresa gestiona desde pequeñas reuniones hasta grandes conferencias, y ofrece servicios adicionales como catering, alquiler de equipos audiovisuales, y gestión de inscripciones. Todos estos servicios son externos y se han ido sumando en el tiempo.

- La empresa debe poder crear, actualizar y cancelar eventos. Cada evento tiene detalles como fecha, ubicación, tipo de evento, y lista de asistentes.
- Las personas o participantes pueden inscribirse en línea para los eventos. Cada participante tiene nombres, apellidos, cédula, género (solo se permite femenino y masculino), número de teléfono y correo electrónico.
- El sistema debe gestionar las inscripciones, incluyendo varias operaciones como confirmar asistencia, cancelar inscripción y manejar listas de espera.
- Para cada evento, se pueden contratar servicios adicionales como catering o equipos audiovisuales, entre otros.
- Estos servicios adicionales tienen costos asociados que deben ser calculados según lo que se necesite. Por ejemplo, cuántos equipos audiovisuales se necesitan, o cuál es la cantidad y tipo de pasapalos o comidas que se requieran.
- El costo del evento dependerá de varios factores, incluido los servicios adicionales contratados.
- Los participantes deben recibir notificaciones sobre cambios en el evento, recordatorios antes del evento, y encuestas de satisfacción después del evento.
- Enviar notificaciones implica lógica de negocio que puede cambiar con cierta frecuencia según las exigencias de cada evento.

Ahora bien, tenga en cuenta las siguientes restricciones a nivel del diseño del modelo de dominio que usted debe considerar obligatorio:

- Existe solamente los siguientes cuatro (4) agregados: ***Evento, Participante, Catering, Audiovisual.***
- Tenga presente que las notificaciones implican lógica de negocio a dos niveles. Es decir, a nivel de ***Enterprise Business Rules*** y de ***Application Business Rules***.

Usted debe identificar *todos los servicios de dominio* necesarios para completar este modelo de dominio en base al enunciado. Para ello debe nombrar el servicio y explicar su responsabilidad justificando por qué es un servicio de dominio.

Sin justificación no hay puntaje completo.

4. DDD + AOP + Arquitectura Hexagonal (7 puntos)

Considere el siguiente dominio de problema:

- En un sistema de compras en línea, los clientes están conformados por su id de usuario, email, nombre y números de teléfono.
- Cuando un cliente hace una compra, se genera una orden de compra conformada por el id de la orden, el monto total de la compra y la fecha.
- Cuando la orden de compra es exitosa, se debe ejecutar la actualización de los puntos de fidelidad del cliente.
- Los puntos de fidelidad del cliente es un tema complejo, es un algoritmo, que puede cambiar en el tiempo.
- Los clientes tienen niveles de fidelidad. El primer nivel es de fidelidad inicial, y luego según su histórico de compras, va subiendo a Silver, Gold, Platinum y Premium+.
- Es importante tener presente que los puntos de fidelidad se calculan en base al histórico de compras, el monto de la compra nueva, el nivel de fidelidad actual del cliente.
- Por razones de eficiencia el histórico de las órdenes de compra se manejan en un sistema aparte de alto desempeño, para favorecer las consultas de una manera eficiente, para mayor rapidez en las respuestas.
- En la capa de aplicación se manejan los errores, y esto es muy importante en este diseño, ya que se pueden generar distintos tipos de excepciones: de dominio, de aplicación y de infraestructura. Pueden existir muchas fallas. Lo que se necesita es **transformar** cualquier excepción que se produzca a un error en código HTTP (40X, 50X, etc.) que es lo que se enviará mediante el API hacia las aplicaciones de frontend que consumen el backend.
- **Importante:** *la transformación de estos errores ocurre a nivel de la capa de aplicación. Usted debe modelar esto de la manera correcta usando todo lo aprendido en el curso. De la capa de aplicación se propaga hacia la capa de infraestructura el error transformado como lo necesitan los controladores que representan los endpoints del API Rest (HTTP).*

Usted debe responder las siguientes cuestiones:

- a. **(0,5 puntos)** Identifique del enunciado **todos los agregados**. Especifique **los *value objects* o entidades** que contienen dichos agregados según el enunciado.
- b. Identifique **el aspecto** que reside en la capa de aplicación.
- c. **(2 punto)** Identifique **los servicios de aplicación**, justificando **la responsabilidad de cada uno**.
- d. **(0,5 puntos)** Identifique **el evento de domino**.
- e. **(1 punto)** Identifique **el servicio de dominio**, justificando **su responsabilidad con claridad**.

(3 puntos) Adicionalmente usted debe **implementar en *TypeScript* el aspecto identificado**. Tenga como referencia que su código debe cumplir con las especificaciones que hemos venido manejando en el curso:

```
interface IService<TService, RService> {
    execute(s: TService): Either<Error, RService>;
}

class Optional<T> {
    hasValue(): boolean { ... }
    getValue(): T { ... }
}

class Either<TLeft, TRight> {
    isLeft(): boolean { ... }
    getLeft(): TLeft { ... }
    isRight(): boolean { ... }
    getRight(): TRight { ... }
    static makeLeft<TLeft, TRight>(value: TLeft) { ... }
    static makeRight<TLeft, TRight>(value: TRight) { ... }
}
```
